

前端通关手册：JavaScript

来自力扣《前端通关手册：JavaScript》

View / MVVM 框架

对比 React、Angular 和 Vue

该问题 Vue 官方在《对比其他框架》中作过说明，整理如下：

- React 和 Vue 都提供了 Virtual Dom
- 提供了响应式（Reactive）和组件化（Composable）的视图组件
- 注意力保持在核心库，而将其他功能如路由和全局状态管理交给相关的库

React 和 Vue 不同点

渲染优化

- React：当组件状态发生变化时，以该组件为根，重新渲染整个组件子树
 - shouldComponentUpdate，采用合适方式，如不可变对象，比较 props 和 state，决定是否重新渲染当前组件
 - 使用 PureComponent：继承自 Component 类，自动加载 shouldComponentUpdate 函数，自动对 props 和 state 浅比较决定是否触发更新
- Vue：自动追踪组件依赖，精确渲染状态改变的组件

HTML 和 CSS

- React：支持 JSX
 - 在构建视图时，使用完整的 JavaScript 功能
 - 开发工具多支持 JSX 语法高亮，类型检查和自动完成
- Vue：提供渲染函数，支持 JSX，但默认推荐 Vue 模版
 - 与书写 HTML 更一致的开发体验
 - 基于 HTML 的模版更容易迁移到 Vue
 - 设计师和新人开发者更容易理解和参与
 - 支持模板预处理器，如 Pug

CSS 作用域

- React：通过 CSS-in-JS 方案，如 styled-components 和 emotion
- Vue：
 - 支持 CSS-in-JS 方案，如 styled-components-vue 和 vue-emotion
 - 单文件组件中 style 标签可选 scoped 属性。支持 Sass \ Less \ Stylus 等 CSS 预处理器，深度集成 CSS Modules

规模

向上扩展：

- React：
 - 路由库和状态管理库，由社区维护支持，生态松散且繁荣
 - 提供脚手架工具，故意作了限制
 - 不允许在项目生成时进行配置

- 只提供一个单页面应用模板
 - 不支持预设配置
- Vue:
 - 路由库和状态管理库，由官方维护支持，与核心库同步更新
 - 提供 CLI 脚手架，可构建项目，快速开发组件的原型
 - 允许在项目生成时配置
 - 提供了各种用途的模板
 - 支持预设配置

向下拓展

- React: 学习曲线陡峭，需要前置知识：JSX ES2015，需要学习构建系统
- Vue: 既支持向上拓展与 React 一样，也支持向下拓展与 jQuery 一样，上手快

原生渲染

- React Native: 使用相同组件模型编写具有本地渲染能力的 APP，跨平台开发
- Weex: 阿里巴巴发起，Apache 基金会孵化，同样支持本地渲染，跨平台开发
 - NativeScript-Vue，基于 Vue.js 构建原生应用的 NativeScript 插件

MobX

- React: 流行的状态管理框架
- Vue: 选择 Vue 比 React + MobX 更合理

Preact 和其它类 React 库

- 难以保证与 React 库 100% 兼容

Vue 和 Angular 相同点

- TypeScript 都支持 TypeScript，支持 类型声明 和 组件装饰器
- 运行时性能，Angular 和 Vue 都很快

Vue 和 Angular 不同点

体积

- Angular 用 AOT 和 tree-shaking 缩小体积
- Vuex + Vue Router (gzip 后 30kB) 比使用优化 angular-cli (~65kB) 小

灵活性

- Vue 提供构建工具，不限制组织应用代码的方式
- Angular 提供构建工具，有相对严格的代码组织规范

学习曲线

- Vue 只需 HTML 和 JavaScript 基础
- Angular 提供 API 和 概念 更多，设计目标针对 大型复杂应用，对新手有难度

如何实现一个组件，前端组件的设计原则是什么？

- 单一原则：一个组件只做一件事
- 通过脑图、结构图，标识组件的 State Props Methods 生命周期，表示层次和数据流动关系
- State 和 Props
 - 扁平化：最多使用一层嵌套，便于对比数据变化，代码更简洁

- 无副作用：State 仅响应事件，不受其他 State 变化影响
- 松耦合
 - 组件应该独立运行，不依赖其它模块
- 配置、模拟数据、非技术说明文档、helpers、utils 与 组件代码分离
- 视图组件只关心 视图，数据获取，过滤，事件处理应在外部 JS 或 父组件 中处理
- Kiss原则 (Keep it Simple Stupid)
 - 不需要 State 时，使用 函数组件
 - 不要传递不需要的 Props
 - 及时抽取复杂组件为独立组件
 - 不要过早优化
- 参考 CSS 的 OOSS 方法论，分离 位置 和 样式，利于实现皮肤
- 考虑 多语言、无障碍 等后期需求

Angular

如何理解 Angular 的依赖注入？

依赖注入 (DI) 是一种重要应用设计模式

- 依赖：类执行功能，所需要服务或对象
- DI 框架会在实例化类时，向其提供这个类声明的依赖项
- 使用 ng generate service 生成 Service 类

分层注入体系

ModuleInjector

- @Injectable 装饰器标记服务 (推荐，利于摇树优化)
 - 可以配置 provider 是 root 或 具体的 NgModule
- @NgModule 装饰器都有 providers 元数据
 - 可以覆盖 @Injectable 中的 provider，来配置多个应用共享的服务非默认提供者

ElementInjector

- 为每个 DOM 元素隐式创建 ElementInjector
- 默认为空
- 在 @Directive 和 @Component 的 providers 属性中配置

注入器继承机制

当组件声明依赖时

- 优先使用它自己的 ElementInjector 来满足依赖
- 组件缺少提供者，请求转发至 父组件 ElementInjector
- 任何 ElementInjector 都找不到，返回发起请求元素
- 在 ModuleInjector 层次结构中查找
- 任何 ModuleInjector 都找不到，引发错误

解析修饰符

- @Optional
 - 声明依赖是可选的，找不到返回 null，无需抛出错误
- @Self
 - 仅查看当前组件或指令的 ElementInjector

- @SkipSelf
 - 跳过当前，查找父级的 ElementInjector
- @Host
 - 当前组件作为注入器树的最后一站

VUE

computed 与 watch 的区别?

computed

- 支持数据缓存
- 内部数据改变也会触发
- 不支持异步，异步无效
- 适用于 一个属性由其他属性计算而来，依赖其他属性的场景

watch

- 不支持数据缓存
- 可以设置一个函数，带有两个参数，新旧数据
- 支持异步
- 监听数据必须是 data 中声明过或者父组件传递过来的 props 中数据
 - immediate：组件加载立即触发回调函数执行
 - deep：深度监听

Vue.nextTick 原理和作用?

- Vue 异步执行 DOM 更新，观察数据变化，开启队列缓冲同一事件循环中所有数据改变
- 同一个 watcher 被多次触发，只会被推入到队列中一次，避免不必要的计算和 DOM 操作
- 在下一个事件循环 tick 中，Vue 刷新队列并执行实际（已去重）工作
- 异步队列使用 Promise.then 和 MessageChannel，不支持环境使用 setTimeout(fn, 0)
- 在需要立即更新 DOM 的场景中使用

Vue3.x 的新特性

API 风格

- Vue2.x: Options API
- Vue3.x: Composition API

生命周期

- 组件生命周期
 - Vue2.x:
 - beforeCreate
 - created
 - beforeMount
 - mounted
 - beforeUpdate
 - updated
 - beforeDestroy
 - destroyed
 - activated

- deactivated
 - errorCaptured
- Vue3.x:
 - setup
 - onBeforeMount
 - onMounted
 - onBeforeUpdate
 - onUpdate
 - onBeforeUnmount
 - onUnmounted
 - onActivated
 - onDeactivated
 - onErrorCaptured
 - onRenderTriggered
 - onRenderTracked
- 指令生命周期
 - Vue2.x:
 - bind
 - inserted
 - update
 - componentUpdated
 - unbind
 - Vue3.x:
 - beforeMount
 - mounted
 - beforeUpdate
 - updated
 - beforeUnmount
 - unmounted

数据

- Vue2.x: data
- Vue3.x
 - ref 基础类型和对象的双向绑定
 - reactive 对象的双向绑定
 - 通过 toRefs 转为 ref

监听

- Vue2.x: watch
- Vue3.x: watchEffect
 - 传入回调函数，不需要指定监听的数据源，自动收集响应式数据
 - 可以从回调函数中，获取数据的最新值，无法获取原始值

slot

- Vue2.x: 通过 slot 使用插槽，通过 slot-scope 给插槽绑定数据
- Vue3.x: v-slot: 插槽名 = 绑定数据名

v-model

- Vue2.x: .async 绑定属性和 update:~ 属性名 事件
- Vue3.x: 无需 .async 修饰

新功能

- Teleport: 适合 Model 实现
- Suspense:
 - 一个组件两个 template
 - 先渲染 #fallback 内容, 满足条件再渲染 #default
 - 适合异步渲染显示加载动画, 骨架屏等
- defineAsyncComponent: 定义异步组件
- 允许多个根节点

性能

- 使用 Proxy 代替 defineProperty 和 数组劫持
- 标记节点类型, diff 时, 跳过静态节点
- 支持 ES6 引入方法, 按需编译
- 配套全新的 Web 开发构建工具 Vite

React

React 生命周期

React 16 前的生命周期

- Mounting
 - componentWillMount
 - render
 - componentDidMount
- Updation
 - props
 - componentWillReceiveProps
 - shouldComponentUpdate
 - componentWillUpdate
 - render
 - componentDidUpdate
 - states
 - shouldComponentUpdate
 - componentWillUpdate
 - render
 - componentDidUpdate
- Unmounting
 - componentWillUnmount

React 16.3 生命周期

- Mounting
 - constructor
 - getDerivedStateFromProps
 - render
 - React 更新 DOM 和 refs

- componentDidMount
- Updation
 - props 变化 → getDerivedStateFromProps
 - setState() → shouldComponentUpdate
 - forceUpdate() → render
 - getSnapshotBeforeUpdate
 - React 更新 DOM 和 refs
 - componentDidUpdate
- Unmounting
 - componentWillUnmount

React 16.4 + 生命周期

- Mounting
 - constructor
 - getDerivedStateFromProps
 - render
 - React 更新 DOM 和 refs
 - componentDidMount
- Updation
 - props 变化 → getDerivedStateFromProps
 - setState() → getDerivedStateFromProps → shouldComponentUpdate
 - forceUpdate() → getDerivedStateFromProps
 - render
 - getSnapshotBeforeUpdate
 - React 更新 DOM 和 refs
 - componentDidUpdate
- Unmounting
 - componentWillUnmount

React 中使用 useEffect 如何清除副作用？

在 useEffect 中返回一个清除函数，名称随意，可以是匿名函数或箭头函数，在清除函数中添加 处理副作用的逻辑，如移除订阅等

```
function component(props) {
  function handleStatusChange(status) { console.log(status.isOnline) }
  useEffect(() => {
    API.subscribe(props.id, handleStatusChange)
  })
  return function cleanup() {
    API.unsubscribe(props.id, handleStatusChange)
  }
}
```

对比 React 和 Vue 的 diff 算法？

(1) 相同点

- 虚拟 Dom 只同级比较，不跨级比较
- 使用 key 标记和复用节点，不建议使用数组索引 index 作为 key

(2) 不同点

- 顺序
 - Vue: 两端到中间
 - React: 从左到右
- 节点元素类型相同, ClassName 不同
 - Vue: 不同类型元素, 删除重新创建
 - React: 相同类型元素, 修改
- 节点类型
 - Vue 3.x: VNode 创建时, 即确定类型

React 中有哪几种类型的组件, 如何选择?

- 无状态组件
 - 更适合函数组件
 - 负责展示
 - 无状态, 复用度高
- 有状态组件
 - 函数组件 + hooks 或 类组件
 - useState 或 声明 state
 - useEffect 或 使用生命周期
- 容器组件
 - 子组件状态提升到此, 统一管理
 - 异步操作, 如数据请求等
 - 提高子组件的复用度
- 高阶组件
 - 接收组件, 返回组件
 - 为原有组件增加新功能和行为
 - 代替 mixins, 避免状态污染
- 回调组件
 - 高阶组件的另一种形式
 - 将组件本身, 通过 props.children 或 prop 属性 传递给子组件
 - 适合不能确定或不关心传给子组件数据的场景, 如路由, 加载组件的实现

CSS

如何实现单行居中, 多行居左?

父级元素:

```
text-align: center;
```

自身元素:

```
text-align: left;  
display: inline-block;
```

如何实现 1px 边框

前置知识:

现代 webkit 内核浏览器提供 私有属性

- -webkit-min-device-pixel-ratio

当前设备的物理像素分辨率与CSS像素分辨率比值的最小值

- -webkit-max-device-pixel-ratio

当前设备的物理像素分辨率与 CSS 像素分辨率比值的最大值

分别可以用标准属性 min-resolution 和 max-resolution 替代

方法一：border-width

```
.border {  
    border: 1px solid;  
}  
@media screen and {min-resolution: 2dppx} {  
    .border {  
        border: 0.5px solid;  
    }  
}  
@media screen and (min-resolution: 3dppx) {  
    .border {  
        border: 0.333333px solid;  
    }  
}
```

方法二：伪类 + transform

```
div::after {  
    content: '';  
    display: block;  
    border-bottom: 1px solid;  
}  
@media only screen and (min-resolution: 2dppx) {  
    div::after {  
        -webkit-transform: scaleY(.5);  
        transform: scaleY(.5);  
    }  
}  
@media only screen and (min-resolution: 3dppx) {  
    div::after {  
        -webkit-transform: scaleY(.33);  
        transform: scaleY(.33);  
    }  
}
```

多方法实现高度 100% 容器

(1) 百分比

```
<style>
html, body {
    height: 100%;
}
div {
    height: 100%;
    background-color: azure;
}
</style>
<div></div>
```

(2) 相对单位

```
<style>
div {
    height: 100vh;
    background-color: azure;
}
</style>
<div></div>
```

(3) calc

```
<style>
html, body{
    height: 100%;
}
div {
    height: calc(100%)
}
</style>
<div></div>
```

多方法实现圆角边框

- 背景图片：绘制圆角边框的图片，4个圆角 + 4个边框的小图片，拼成圆角边框
- border-radius: 5px
- clip-path: inset(0 round 5px)

多方法实现文字描边

- text-shadow: 0 0 1px black;
- -webkit-text-stroke: 1px black;
- position: relative / position: absolute 子绝父相

JavaScript

ES3

a = [], a.push(...[1, 2, 3]) , a = ?

a = [1, 2, 3], 考核点如下:

- [].push: 调用数组 push 方法
- apply:
 - 第一参数: 指定 push 执行时的 this, 即正在调用 push 的对象为数组 a
 - 第二参数: 传入数组或类数组对象作为参数数组, 展开作为 push 的参数列表
- push的语法: 支持将一个或多个元素添加到数组末尾

```
arr.push(element1, ..., elementN)
```

综上, 题目等同于

```
a.push(1, 2, 3) // [1, 2, 3]
```

a = ?, a==1 && a==2 && a==3 成立

== 会触发隐式转换, === 不会

对象转字符串

- 先尝试调用对象的 toString()
- 对象无 toString() 或 toString 返回非原始值, 调用 valueOf() 方法
 - 将该值转为字符串, 并返回字符串结果
- 否则, 抛出类型错误

对象转数字

- 先尝试调用对象的 valueOf(), 将返回原始值转为数字
- 对象无 valueOf() 或 valueOf 返回不是原始值, 调用 toString() 方法, 将返回原始值转为数字
- 否则, 抛出类型错误

对象转布尔值

- True

代码

```
const a = {
  count: 0,
  valueOf() {
    return ++this.count
  }
}
```

数组

隐式转换会调用数组的 join 方法, 改写此方法

```
const a = [1, 2, 3]
a.join = a.shift
```

null == undefined 结果

- 比较 null 和 undefined 的时候，不能将 null 和 undefined 隐式转换，规范规定结果为相等

常见的类型转换

类型	值	to Boolean	to Number	to String
Boolean	true	true	1	"true"
Boolean	false	false	0	"false"
Number	123	true	123	"123"
Number	Infinity	true	Infinity	"Infinity"
Number	0	false	0	"0"
Number	NaN	false	NaN	"NaN"
String	""	false	0	""
String	"123"	true	123	"123"
String	"123abc"	true	NaN	"123abc"
String	"abc"	true	NaN	"abc"
Null	null	false	0	"null"
Undefined	undefined	false	NaN	"undefined"
Function	function() {}	true	NaN	"function(){}"
Object	{}	true	NaN	"[object Object]"
Array	[]	true	0	""
Array	["abc"]	true	NaN	"abc"
Array	["123"]	true	123	"123"
Array	["123", "abc"]	true	NaN	"123, abc"

对比 get 和 Object.defineProperty

相同点

都可以定义属性被查询时的函数

不同点

在 classes 内部使用

- get 属性将被定义到 实例原型
- Object.defineProperty 属性将被定义在 实例自身

对比 escape encodeURI 和 encodeURIComponent

escape

对字符串编码

ASCII 字母、数字 @ * / + - _ . 之外的字符都被编码

encodeURI

对 URL 编码

ASCII 字母、数字 @ * / + 和 ~ ! # \$ % & () = , ; ? - _ . ' 之外的字符都被编码

encodeURIComponent

对 URL 编码

ASCII 字母、数字 ~ ! * () - _ . ' 之外的字符都被编码

事件

事件传播的过程

事件冒泡

- DOM0 和 IE 支持 (DOM1 开始是 W3C 规范)
- 从事件源向父级，一直到根元素 (HTML)
- 某个元素的某类型事件被触发，父元素同类型事件也会被触发

事件捕获

- DOM2 支持
- 从根元素 (HTML) 到事件源
- 某个元素的某类型事件被触发，先触发根元素，再向子一级，直到事件源

事件流

- 事件的流向：事件捕获 → 事件源 → 事件冒泡

阻止事件冒泡

- 标准：event.stopPropagation()
- IE：event.cancelBubble = true

Event Loop 的执行顺序？

宏任务

- Task Queue
- 常见宏任务：setTimeout、setInterval、setImmediate、I/O、script、UI rendering

微任务

- Job Queue
- 常见微任务：
 - 浏览器：Promise、MutationObserver
 - Node.js：process.nextTick

执行顺序

- 首先执行同步代码，宏任务
- 同步栈为空，查询是否有异步代码需要执行

- 执行所有微任务
- 执行完，是否需要渲染页面
- 重新开始 Event Loop，执行宏任务中的异步代码

为什么 Vue.\$nextTick 通常比 setTimeout 优先级高，渲染更快生效？

- Vue.\$nextTick 需要异步执行队列，异步函数的实现优先使用
 - Promise、MutationObserver、setImmediate
 - 都不兼容时，使用 setTimeout
- Promise、MutationObserver、setImmediate 是微任务
- setTimeout、UI rendering 是宏任务
- 根据执行顺序
 - Promise、MutationObserver、setImmediate 创建微任务，添加到当前宏任务微任务队列。队列任务执行完，如需渲染，即可渲染页面
 - setTimeout 创建宏任务，如果此时正在执行微任务队列，需要等队列执行完，渲染一次后，重新开始 Event Loop，执行宏任务中的异步代码后再渲染

ES6 新特性

列举 ES6、ES7、ES8、ES9、ES10 新特性

ES6

- let 和 const
- Promise
- Class
- 箭头函数
- 函数参数默认值
- 模版字符串
- 解构赋值
- 展开语法
 - 构造数组，调用函数时，将 数组表达式 或 string 在语法层面展开
- 对象属性缩写
 - 键名和键值相同
 - 函数省略 function
- 模块化

ES7

- includes()
- 指数操作符**

ES8

- async/await
- Object.values()
- Object.entries()
- Object.getOwnPropertyDescriptors()
- 填充字符串到指定长度：padStart、padEnd

- `ShareArrayBuffer` 和 `Atomics`，共享内存位置读取和写入
- 函数最后参数有尾逗号，与数组和对象保持一致

ES9

- 异步迭代: `for await (let i of array)`
- `Promise.finally()`
- 展开语法
 - 构造字面量对象时，将对象按照键值对展开，克隆属性或浅拷贝
- 非转义序列的模版字符串
- 正则表达式
 - 命名捕获组:

```
const match = /(?!<year>\d{4})/.exec('2022')
console.log(match.groups.year) // 2022
```

- 反向断言:
 - 肯定反向断言

```
const match = /^(?<=\d)\d+/.exec('a123')
console.log(match[0]) // 123
```

- 否定反向断言

```
const match = /^(?!<\d)\d+/.exec('a123')
console.log(match[0]) // 123
```

- `dotAll` 模式: 增加 `s` 修饰符，让 `.` 可以匹配换行符
- Unicode 转义: 在正则表达式中本地访问 Unicode 字符属性不被允许

```
/\p{...}/u.test('π')
/\P{...}/u.test('π')
```

- 非转义序列的模版字符串
 - `\u`unicode转义
 - `\x`十六进制转义
 - `\`后跟数字，八进制转义

ES10

- `JSON.stringify`
 - `\ud800` 到 `\udfff` 单独转换，返回转义字符串
 - `\ud800` 到 `\udfff` 成对转换，对应字符存在，返回字符。不存在，返回转义字符串
- `flat` 和 `flatMap`
- `trimStart` 和 `trimEnd` 去除字符串首尾空白字符
- `Object.fromEntries()`传入键值对列表，返回键值对对象
- `Symbol.prototype.description`

```
const sym = Symbol('description')
sym.description // description
```

- String.prototype.matchAll 返回包含所有匹配正则表达式和分组捕获结果的迭代器
- Function.prototype.toString() 返回精确字符，包括空格和注释
- 修改 catch 绑定
- 新基本数据类型 BigInt
- globalThis
- import()

ES11+

- String.prototype.replaceAll
- Promise.any
 - 一个 resolve 返回第一个 resolve 状态
 - 所有 reject 返回请求失败
- WeakRefs
 - 通过 WeakMap、WeakSet 创建
 - 创建对象的弱引用：该对象可被回收，即使它仍被引用

逻辑运算符赋值表达式

- ||=

```
a ||= b // 若 a 不存在，则 a = b
```

- &&=

```
a &&= b // 若 a 存在，则 a = b
```

- ??=

```
a ??= b // 若 a 为 null 或 undefined，则 a = b
```

- 访问对象未定义属性
 - ?.

```
const a = {}
a?.b?.c // undefined, 不报错
```

- 数字分隔符

```
123_1569_9128 // 12315699128
```

变量

列举类数组对象

(1) 定义

- 拥有 length 属性
- 若干索引属性的任意对象

(2) 举例

- NodeList
- HTML Collections
- 字符串
- arguments
- \$ 返回的 jQuery 原型对象

(3) 类数组对象转数组

- 新建数组，遍历类数组对象，将每个元素放入新数组
- Array.prototype.slice.call(ArrayLike) 或 [].slice.call(ArrayLike)
- Array.from(ArrayLike)
- apply 第二参数传入，调用数组方法

```
Array.prototype.push.apply(null, ArrayLike)
```

闭包

什么是闭包？

闭包是由函数以及声明该函数的词法环境组合而成

- 一个函数和对其周围状态（词法环境）的引用捆绑在一起（或者说函数被引用包围），这样的组合就是闭包
- 可以在内层函数中访问到其外层函数的作用域
- 每当创建一个函数，闭包就会在函数创建的同时被创建出来

什么是词法？

词法，英文 lexical，词法作用域根据源代码**声明变量的位置**来确定变量在何处可用

嵌套函数可访问声明于它们外部作用域的变量

什么是函数柯里化？

函数调用：一个参数集合应用到函数

部分应用：只传递部分参数，而非全部参数

柯里化（curry）：使函数理解并处理部分应用的过程

- 保存调用 curry 函数时传入的参数，返回一个新函数
- 结果函数在被调用后，要让新的参数和旧的参数一起应用到入参函数

bind / apply / call / new

手写 bind

- 第一个参数接收 this 对象
- 返回函数，根据使用方式
 - 直接调用
 - 改变 this 指向
 - 拼接参数
 - 调用函数
 - 构造函数
 - 不改变 this 指向，忽略第一参数
 - 拼接参数
 - new 函数

```
Function.prototype.myBind = function(_this, ...args) {
  const fn = this
  return function F(...args2) {
    return this instanceof F ? new fn(...args, ...args2)
      : fn.apply(_this, args.concat(args2))
  }
}
//使用
function Sum (a, b) {
  this.v= (this.v || 0)+ a + b
  return this
}
const NewSum = Sum.myBind({v: 1}, 2)
NewSum(3) // 调用: {v: 6}
new NewSum(3) // 构造函数: {v: 5} 忽略 myBind 绑定this
```

手写 call

- 第一参数接收 this 对象
- 改变 this 指向：将函数作为传入 this 对象的方法
- 展开语法，支持传入和调用参数列表
- 调用并删除方法，返回结果

```
Function.prototype.myCall = function(_this, ...args) {
  if (!_this) _this = Object.create(null)
  _this.fn = this
  const res = _this.fn(...args)
  delete _this.fn
  return res
}
// 使用
function sum (a, b) {
  return this.v + a + b
}
sum.myCall({v: 1}, 2, 3) // 6
```

手写 apply

- 第一参数接收 this 对象
- 改变 this 指向：将函数作为传入 this 对象的方法
- 第二个参数默认数组
- 展开语法，支持调用参数列表
- 调用并删除方法，返回结果

```
Function.prototype.myApply = function(_this, args = []) {  
  if (!_this) _this = Object.create(null)  
  _this.fn = this  
  const res = _this.fn(...args)  
  delete _this.fn  
  return res  
}  
// 使用  
function sum (a, b) {  
  return this.v + a + b  
}  
sum.myApply({v: 1}, [2, 3]) // 6
```

手写 new

- 第一参数作为构造函数，其余参数作为构造函数参数
- 继承构造函数原型创建新对象
- 执行构造函数
- 结果为对象，返回结果，反之，返回新对象

```
function myNew(...args) {  
  const Constructor = args[0]  
  const o = Object.create(Constructor.prototype)  
  const res = Constructor.apply(o, args.slice(1))  
  return res instanceof Object ? res : o  
}  
// 使用  
function P(v) {  
  this.v = v  
}  
const p = myNew(P, 1) // P {v: 1}
```

手写防抖

- 声明定时器
- 返回函数
- 一定时间间隔，执行回调函数
- 回调函数
 - 已执行：清空定时器
 - 未执行：重置定时器

```
function debounce(fn, delay) {
  let timer = null
  return function (...args) {
    if (timer) clearTimeout(timer)
    timer = setTimeout(() => {
      timer = null
      fn.apply(this, args)
    }, (delay + '') || 0 || 1000 / 60)
  }
}
```

手写节流

- 声明定时器
- 返回函数
- 一定时间间隔，执行回调函数
- 回调函数
 - 已执行：清空定时器
 - 未执行：返回

```
function throttle(fn, interval) {
  let timer = null
  return function (...args) {
    if (timer) return
    timer = setTimeout(() => {
      timer = null
      fn.apply(this, args)
    }, (interval + '') || 0 || 1000 / 60)
  }
}
```

原型链

对比各种继承？

(1) 原型链继承

子类原型指向父类实例

```
function Parent() {}
function Child() {}
Child.prototype = new Parent()
const child = new Child()
```

好处：

- 子类可以访问到父类新增原型方法和属性

坏处：

- 无法实现多继承
- 创建实例不能传递参数

(2) 构造函数

```
function Parent() {}
function Child(...args) {
  Parent.call(this, ...args) // Parent.apply(this, args)
}
const child = new Child(1)
```

好处:

- 可以实现多继承
- 创建实例可以传递参数

坏处:

- 实例并不是父类的实例，只是子类的实例
- 不能继承父类原型上的方法

(3) 组合继承

```
function Parent() {}
function Child(...args) {
  Parent.call(this, ...args)
}
Child.prototype = Object.create(Parent.prototype)
Child.prototype.constructor = Child
const child = new Child(1)
```

好处:

- 属性和原型链上的方法都可以继承
- 创建实例可以传递参数

(4) 对象继承

- Object.create

```
const Parent = { property: 'value', method() {} }
const child = Object.create(Parent)
```

- create

```
const Parent = { property: 'value', method() {} }
function create(obj) {
  function F() {}
  F.prototype = obj
  return new F()
}
const child = create(Parent)
```

好处:

- 可以继承属性和方法

坏处:

- 创建实例无法传递参数
- 传入对象的属性有引用类型，所有类型都会共享相应的值

(5) 寄生组合继承

```
function Parent() {}
function Child(...args) {
  Parent.call(this, args)
}
function create(obj) {
  function F() {}
  F.prototype = obj
  return F()
}
function extend(Child, Parent) {
  const clone = create(Parent.prototype)
  clone.constructor = Child
  Child.prototype = clone
}
extend(Child, Parent)
const child = new Child(1)
```

(6) ES6继承

```
class Parent {
  constructor(property) {
    this.property = property
  }
  method() {}
}
class Child extends Parent {
  constructor(property) {
    super(property)
  }
}
```

模块化

webpack 中 loader 和 plugin 的区别?

loader

- 在打包前或期间调用
- 根据不同匹配条件处理资源
- 调用顺序与书写顺序相反
- 写在前面的接收写在后面的返回值作为输入值

plugin

- 基于 Tapable 实现
- 事件触发调用，监听 webpack 广播的事件
- 不同生命周期改变输出结果

如何自定义一个 webpack 插件?

- 声明一个自定义命名的类或函数
- 在原型中新增 apply 方法
- 声明由 Compiler 模块暴露的生命周期函数
- 使用 webpack 提供的 API 或 自行处理内部实例的数据
- 处理完成后，调用 webpack 提供的回调函数

示例:

实现一个 MyPlugin, 获取指定图片, 新增静态资源到本地

```
class MyPlugin { // 声明类
  apply(compiler) { // 新增 apply 方法
    // 声明生命周期函数
    compiler.hooks.additionalAssets.tapAsync('MyPlugin', (compilation,
callback) => {
      download('https://img.shields.io/npm/v/webpack.svg', res => {
        if (res.status === 200) {
          // 调用 API
          compilation.assets['webpack-version.svg'] = toAsset(res)
          // 异步处理后, 调用回调函数
          callback()
        } else {
          callback(new Error('[webpack-example-plugin] Unable to
download the image'))
        }
      })
    })
  }
}
```

对比 import、import() 和 require

-	import	import()	require
规范	ES6Module	ES6Module	CommonJS
执行阶段	静态 编译阶段	动态 执行阶段	动态 执行阶段
顺序	置顶最先	异步	同步
缓存	√	√	√
默认导出	default	default	直接赋值
导入赋值	解构赋值, 传递引用	在then方法中解构赋值, 属性值是仅可读, 不可修改	基础类型 赋值, 引用类型 浅拷贝

如何实现一个深拷贝, 要点是什么?

JSON

- 不支持 Symbol, BigInt, Function
- 不支持 循环引用
- 丢失值为 undefined 的键

```
function deepCopy(o) {
  return JSON.parse(JSON.stringify(o))
}
```

递归

- 递归处理 引用类型
- 保持数组类型

```
function deepCopy(target) {
  if (typeof target === 'object') {
    const newTarget = Array.isArray(target) ? [] : Object.create(null)
    for (const key in target) {
      newTarget[key] = deepCopy(target[key])
    }
    return newTarget
  } else {
    return target
  }
}
```

递归

- 哈希表 Map 支持 循环引用
 - Map 支持引用类型数据作为键

```
function deepCopy(target, h = new Map) {
  if (typeof target === 'object') {
    if (h.has(target)) return h.get(target)
    const newTarget = Array.isArray(target) ? [] : Object.create(null)
    for (const key in target) {
      newTarget[key] = deepCopy(target[key], h)
    }
    h.set(target, newTarget)
    return newTarget
  } else {
    return target
  }
}
```

递归

- 哈希表 WeakMap 代替 Map
 - WeakMap 的键是弱引用，告诉 JS 垃圾回收机制，当键回收时，对应 WeakMap 也可以回收，更适合大量数据深拷贝

```
function deepCopy(target, h = new WeakMap) {
  if (typeof target === 'object') {
    if (h.has(target)) return h.get(target)
    const newTarget = Array.isArray(target) ? [] : Object.create(null)
    for (const key in target) {
      newTarget[key] = deepCopy(target[key], h)
    }
    h.set(target, newTarget)
    return newTarget
  } else {
    return target
  }
}
```

可以继续完善点

- 递归改迭代, 预防栈溢出
- 支持 null、Symbol、BigInt、布尔对象、正则对象、Date对象等 深拷贝
- 使用 while / for 代替遍历数组, 使用 Object.keys() 代替遍历对象

0.1 + 0.2 !== 0.3 的原因, 如何解决?

- 计算机采用二进制表示十进制
 - 将十进制的 0.1 转为二进制

```
0.1 * 2 = 0.2 0
0.2 * 2 = 0.4 0
0.4 * 2 = 0.8 0
0.8 * 2 = 1.6 1
0.6 * 2 = 1.2 1
0.2 * 2 = 0.4 0
...
```

用科学计数法表示: $2^{-4} * 1.10011(0011)$

- 将十进制的 0.2 转为二进制

```
0.2 * 2 = 0.4 0
0.4 * 2 = 0.8 0
0.8 * 2 = 1.6 1
0.6 * 2 = 1.2 1
0.2 * 2 = 0.4 0
...
```

用科学计数法表示: $2^{-3} * 1.10011(0011)$

- JS 采用 IEEE 754 双精度版本 (64位)
 - 64位 = 符号位 (1位) + 整数 (11位) + 小数 (52位)
 - 小数超出 52位, 四舍五入
 - 0.1: $2^{-4} * 1.10011(0011 * 12)01$
 - 0.2: $2^{-3} * 1.10011(0011 * 12)010$
 - 0.1 + 0.2: 2^{-3} 后面部分相加

```
0.1100110011001100110011001100110011001100110011001100110011001101
+ 1.1001100110011001100110011001100110011001100110011001100110011010
-----
= 10.011001100110011001100110011001100110011001100110011001100111
```

- 超出52位, 四舍五入: $2^{-2} * 1.0011(0011 * 12)01$
- 转换为十进制: $2^{-2} + 2^{-5} + 2^{-6} + 2^{-9} + 2^{-10} + 2^{-13} + 2^{-14} + \dots + 2^{-49} + 2^{-50} + 2^{-52}$

```
let q = -2, sum = 2 ** -2
while ((q -= 3) >= -50) sum += 2 ** q + 2 ** --q
sum += 2 ** -52 // 0.30000000000000004
```

- $2^{-2} * 1.0011(0011 * 12)01 = 0.30000000000000004 > 0.3$
- 如何解决 $0.1 + 0.2 !== 0.3$?

- Number.EPSILON

- 表示 1 与 Number 间可表示的大于 1 的最小的浮点数之间的差值
- 接近于 Math.max(2, -52), Number.EPSILON !== Math.max(2, -52)

```
const equal = (a, b, c) => {  
  return Math.abs(c - a - b) < Number.EPSILON  
}
```

- toFixed(digits) digits 属于 0 到 20 间，默认 0，实际支持更大范围
 - 使用定点表示法来格式化一个数值，返回给定数字的字符串

```
const equal = (a, b, c) => { // 产生 0.1 + 0.2 === 0.333 结果  
  return (a + b).toFixed(1) === c.toFixed(1)  
}
```

- 小数转整数运算

```
const sum = (a, b) => {  
  const len = Math.max((a + '').split('.')[1].length, (b + '').split('.')[1].length)  
  return (a * len + b * len) / len  
}
```

使用bignumber.js、bigdecimal.js 等JS 库

正则表达式

实现 trim ， 去除首尾空格

方法一：ES5

```
''.trim()
```

方法二：正则

```
''.replace(/^\s+|\s+$/g, '')
```

匹配查询字符串

```
/**  
 * 获取指定的 querystring 中指定 name 的 value  
 * @param {String} name 查询名  
 * @param {String} querystring 查询到的值  
 * @return {String|undefined} 查询到返回String，没有返回undefined  
 */  
function query (name, querystring) {  
  if (name && name.match(/(?:http[s]?|^\s)\{2\}.*?(?:\s|\/)/)) name =  
    encodeURIComponent(name) // 仅编码url  
  const r = (querystring + '').match(new RegExp('(?:^|&)\\s*' + name +  
    '\\s*=\\s*(.*?)\\s*(?:&|$)'))  
  return r === null ? undefined : decodeURIComponent(r[1])  
}
```

异步编程

如何终止Promise

Promises / A+ 标准

- 原 Promise 对象的状态与新对象保持一致
- 返回一个pending状态的 Promise 对象，后续的函数都不会调用

Promise.race 竞速

- 其中一个 Promise 先到达resolve状态，其他 Promise 不会执行

抛出错误

- 其中一个 Promise 抛出一个错误后，错误信息将沿着 链路 向后传递，直至被捕获
- 错误被捕获前的函数不会被调用

async/await 的错误捕获

- try catch
 - await 函数外，包裹 try {}，当 catch (error) 时，捕获错误
- catch
 - async 函数返回的 Promise，调用 then 方法，然后 catch(error)
- 错误优先原则

```
function async get(data) {
  return new Promise((resolve, reject) => {
    setTimeout(() => data ? resolve('res') : reject('error'), 1000)
  })
}2.
function done (data) {
  return get(data).then(res => [null, res]).catch(err => [err, null])
}
[error, res] = await done()
```

手写 Promise

```
const PENDING = 'pending'
const RESOLVED = 'resolved'
const REJECTED = 'rejected'
function Promise(callback) {
  this.state = PENDING
  this.value = null
  this.resolvedCallbacks = []
  this.rejectedCallbacks = []

  callback(value => {
    if (this.state === PENDING) {
      this.state = RESOLVED
      this.value = value
      this.resolvedCallbacks.map(callback => callback(value))
    }
  }, value => {
    if (this.state === PENDING) {
      this.state = REJECTED
```

```

        this.value = value
        this.rejectedCallbacks.map(callback => callback(value))
    }
    })
}
Promise.prototype.then = function (onFulfilled = () => {}, onRejected = () => {})
{
    if (this.state === PENDING) {
        this.resolvedCallbacks.push(onFulfilled)
        this.rejectedCallbacks.push(onRejected)
    }
    if (this.state === RESOLVED) {
        onFulfilled(this.value)
    }
    if (this.state === REJECTED) {
        onRejected(this.value)
    }
}
}

```

实现异步请求的方式

AJAX -Asynchronous JavaScript and XML

```

const xhr = new XMLHttpRequest()
xhr.open('GET', 'https://leetcode-cn.com', true)
xhr.responseType = 'json'
xhr.onreadystatechange = function() {
    if (xhr.readyState === 4 && xhr.status === 200) {
        console.log(xhr.response)
    }
}
xhr.ontimeout = function() {
    console.log('超时')
}
xhr.setRequestHeader('Content-Type', 'application/json')
xhr.send({ requestBody: 1 })

```

\$.ajax

```

$.ajax({
    url: 'https://leetcode-cn.com',
    data: {
        requestBody: 1
    },
    success(data) {
        console.log(data)
    }
}).done(data) { // jQuery 1.5+ 支持
    console.log(data)
}

```

Axios

```
axios({
  url: 'https://leetcode-cn.com',
  data: {
    requestBody: 1
  }
}).then(data => console.log(data))
```

Fetch

```
fetch('https://leetcode-cn.com', {
  requestBody: 1
}).then(res => res.json()).then(res => res)
```

性能

JavaScript 垃圾回收机制

- 内存分配回收是自动的，垃圾回收器定时找出不再使用的数据，释放内存空间

两种回收检测模式

引用计数：

清除没有任何引用指向的数据。无法处理循环引用。IE6使用

标记清除：

标记可达数据，清除不可达数据。可处理循环引用。现代浏览器广泛使用

- 从根出发：包括 全局变量、本地函数的局部变量、参数、调用链上其它函数的局部变量和函数等
- 标记相连的对象为可达和访问过
- 直到引用链上没有未访问过的对象为止
- 删除没有被标记过，即不可达对象

标记清除的优化：

标记清除存在 内存不连续，回收效率低，偶尔卡顿 的缺点

- 只在CPU空闲时进行
- 分代回收：
 - 新生代：存活时间短，新生或经过一次垃圾回收的对象
 - 复制：复制 From 的可达对象 到 To 空间，释放 不可达对象
 - 晋升：复制时，To 空间使用超过 25%，晋升到 老年代
 - 交换：交换 From 和 To 空间
 - 老年代：存活时间长，经过一次被晋升或多次垃圾回收的对象
 - 标记清除
 - 标记整理：清除阶段先整理，将可达对象连续放置一起，再释放之外的内存
 - 增量标记：用增量标记代替全暂停，在回收间歇执行应用逻辑，避免卡顿

详解标记整理算法

- 标记完成
- 存活对象向内存空间一端移动
- 移动完成，清理掉边界外的所有内存

前端常见的内存溢出途径，如何避免？

占用内存且无法被垃圾回收机制回收对象，容易导致内存溢出（泄漏），让应用程序占用本应回收或不需要占用的内存

意外的全局变量：

全局变量是标记清除中的「根」，除非定义为空 或 重新分配，不可回收

- 非严格模式下，没有使用 var let const 声明的变量
- 挂载在 this 下的属性

避免：尽量不使用全局变量，在 JavaScript 头部使用严格模式

未清除的计时器

- setInterval 自身及其回调函数内的对象，即使不被引用，也需要计时器停止才能清除

避免：使用 requestAnimationFrame / setTimeout + 递归 代替 setInterval，并设置边界

删除不完全的对象

- addEventListener 监听的对象已不可达，但监听没有移除。现代浏览器会自动移除
- JS 中引用了 DOM 对象，对象已从 DOM Tree 中移除，但 JS 中依旧保持引用

避免：

- 移除对象前，移除监听。需大量监听对象，使用事件代理监听父元素
- 移除 DOM 后，设置引用该 DOM 的变量为空

闭包中未使用函数引用了可从根访问的父级变量

```
var global = 0
function closure () {
  let fromGlobal = global // 引用全局变量 global
  function unused () { // 闭包内不使用的函数
    if (fromGlobal) return // 引用父级变量 fromGlobal，导致 unused 占用内存
  }
  global = {} // 每次调用闭包 global 都会重新赋值
  /** 避免 **/
  fromGlobal = null
  closure()
}
```

算法

位运算

什么是原码、反码、补码？

原码、反码和补码，均有 符号位 和 数值位 两部分

符号位：用 0 表示正，用 1 表示负

在计算机系统中

- 数值一律用补码来表示和存储，好处
 - 符号位和数值位统一处理
 - 加法和减法统一处理

正数

- 原码：符号位 0
- 反码：与原码相同
- 补码：与原码相同

负数

- 原码：符号位 1
- 反码：符号位不变，数值位取反
- 补码：符号位不变，数值位取反 +1

0

- 原码：+0 0000 0000 -0 1000 0000
- 反码：+0 0000 0000 -0 1111 1111
- 补码：+0 0000 0000 -0 0000 0000 相同

位运算求绝对值？

数在计算机中，用补码表示

负数的补码 = 负数的原码 → 符号位不变，其余位取反，+1

```
-2
原码: 1000 0010
反码: 1111 1101
补码: 1111 1110 (反码加一, 计算机实际存储的值)
取反: 0000 0001
加一: 0000 0010 = 2
```

解法一：根据符号位，正数返回本身，负数返回 取反 + 1

```
const abs = x => {
  const y = x >>> 31 // 看符号位
  return y === 0 ? x : ~x + 1
}
```

解法二：任何数与 -1 异或 ^ 运算，相当于取反。任何数与 0 异或 ^ 运算，还是本身

```
-1
原码: 1000 0001
反码: 1111 1110
补码: 1111 1111
```

无符号右移

```
const abs = x => {  
  const y = x >>> 31  
  return (x ^ -y) + y  
}
```

有符号右移

```
const abs = x => {  
  const y = x >> 31  
  return (x ^ y) - y  
}
```

数组

去除数组中的指定元素

输入: a = ['1', '2', '3', '4', '5', '6'] target = '4'

输出: a = ['1', '2', '3', '5', '6']

方法一:

```
const removeByValue = (a, target) => {  
  for (let i = 0; i < a.length; i++) {  
    if (target === a[i]) {  
      a.splice(i, 1)  
    }  
  }  
  return a  
}
```

方法二:

```
const removeByValue = (a, target) => a.splice(a.findIndex(v => v === target), 1)
```

方法三:

```
const removeByValue = (a, target) => {  
  let j = 0, i = -1  
  while (++i < a.length) {  
    if (a[i] === target) i++  
    a[j++] = a[i]  
  }  
  a.length--  
  return a  
}
```


数组去重方法

```
function unique (arr) {  
  return Array.from(new Set(arr))  
}
```

- Set 去重 + 扩展运算符 ...

```
function unique (arr) {  
  return [...new Set(arr)]  
}
```

- Object

```
function unique (arr) {  
  const h = Object.create(null)  
  arr.forEach(v => h[v] = 1)  
  return Object.keys(h).map(v => v | 0)  
}
```

- Map

```
function unique (arr) {  
  const h = new Map  
  arr.forEach(v => h.set(v, 1))  
  return Array.from(h.keys())  
}
```

- for 循环 + splice

```
function unique (arr) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] === arr[j]) {  
        arr.splice(j, 1)  
        j--  
      }  
    }  
  }  
  return arr  
}
```

- Sort 排序 + 相邻相同 splice

```
function unique (arr) {
  arr.sort()
  for (let i = 0; i < arr.length - 1; i++) {
    if (arr[i] === arr[i + 1]) {
      arr.splice(i, 1)
      i--
    }
  }
  return arr
}
```

- filter + indexOf

```
function unique (arr) {
  return arr.filter((v, index, ar) => ar.indexOf(v) === index)
}
```

- filter + hasOwnProperty

```
function unique (arr) {
  const h = {} // 注意只有 {} 才有 hasOwnProperty
  return arr.filter(v => !h.hasOwnProperty(v) && (h[v] = 1))
}
```

- indexOf + 辅助数组

```
function unique (arr) {
  const r = []
  arr.forEach(v => r.indexOf(v) === -1 && r.push(v))
  return r
}
```

- includes + 辅助数组

```
function unique (arr) {
  const r = []
  arr.forEach(v => !r.includes(v) && r.push(v))
  return r
}function unique (arr) {
  const r = []
  arr.forEach(v => !r.includes(v) && r.push(v))
  return r
}
```

判断一个对象是不是数组 Array

- isPrototypeOf
 - 测试一个对象是否在另一个对象的原型链上
 - prototype 不可省略

```
function isArray(o) {
  return Array.prototype.isPrototypeOf(o)
}
```

- instanceof
 - 用于检测构造函数的 prototype 属性是否出现在某个实例对象的原型链上

```
function isArray(o) {  
  return o instanceof Array  
}
```

- Array.isArray
 - 用于确定传递的值是否是一个 Array

```
function isArray(o) {  
  return Array.isArray(o)  
} *
```

- Object.prototype.toString
 - 方法返回一个表示对象的字符串

```
function isArray(o) {  
  return Object.prototype.toString.call(o) === '[object Array]'  
}
```

移动零

问题

给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序

示例：

```
输入：[0, 1, 0, 3, 12]  
输出：[1, 3, 12, 0, 0]
```

说明：

- (1) 必须在原数组上操作，不能拷贝额外的数组
- (2) 尽量减少操作次数

解析

(1) 辅助数组

```
const moveZeros = a => {  
  const tmp = new Uint32Array(a.length)  
  for (let i = 0, j = 0; i < a.length; i++) if (a[i]) tmp[j++] = a[i]  
  return tmp  
}
```

(2) 双指针交换

```
const moveZeros = a => {
  let i = j = -1
  while (++i < a.length) if (a[i]) a[++j] = a[i]
  while (++j < a.length) a[j] = 0
  return a
}
```

(3) Sort 排序

```
const moveZeros = a => a.sort((a, b) => -(b === 0 ^ 0))
```

空间复杂度为 $O(1)$ 的中序遍历（莫里斯）遍历

```
const isValidBST = (root, l = -Infinity) {
  while(root) {
    if (root.left) {
      const p = root.left
      while (p.right && p.right !== root) p = p.right
      if (p.right === null) p.right = root, root = root.left
      else {
        root = root.right
        p.right = null
      }
    } else {
      root = root.right
    }
  }
}
```

递归

求最大公约数

辗转相除法（又称 欧几里得算法）

- 递归

```
function gcd(a, b) {
  const t = a % b
  if (t === 0) return b
  return gcd(b, t)
}
```

- 迭代

```
function gcd(a, b) {
  let t
  while (t = a % b) {
    a = b
    b = t
  }
  return b
}
```

更相减损法 (又称 九章算术)

- 递归

```
function gcd(a, b) {
  if (a === b) return b
  a > b ? a -= b : b -= a
  return gcd(a, b)
}
```

排序

插入排序

```
const sort = a => {
  for (let i = 1; i < a.length; i++)
    for (let j = i; j-- && a[j + 1] < a[j];)
      [a[j + 1], a[j]] = [a[j], a[j + 1]]
  return a
}
```

快速排序

```
const sort = (a, s = 0, e = a.length - 1) => {
  if (s >= e) return
  let l = s, r = e
  while (l < r) {
    while (l < r && a[r] >= a[s]) r--
    while (l < r && a[l] <= a[s]) l++
    [a[l], a[r]] = [a[r], a[l]]
  }
  [a[l], a[s]] = [a[s], a[l]]
  sort(a, s, l - 1)
  sort(a, l + 1, e)
  return a
}
```

归并排序

```
const sort = (a, l = 0, r = a.length - 1) => {
  if (l === r) return
  const m = l + r >>> 1, t = []
  sort(a, l, m)
  sort(a, m + 1, r)
  let p1 = l, p2 = m + 1, i = 0

  while (p1 <= m || p2 <= r) t[i++] = p2 > r || p1 <= m && a[p1] < a[p2] ? a[p1++]
    : a[p2++]
  for (i = 0; i < t.length; i++) a[l + i] = t[i]
  return a
}
```

冒泡排序

```
const sort = (a) => {
  for(let i = 0; i < a.length - 1; i++)
    for (let j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]]
  return a
}
```

全栈

Node.js

express 中 next 的作用？

next 是 中间件函数的第三参数

- 执行 next(), 控制权交给下个中间件
- 不执行
 - 终止请求
 - 挂起请求

对比 express 和 koa?

- Handler 处理方式
 - Express: 回调函数
 - Koa: Async / Await
- 中间件
 - Express: 同一线程, 线性传递
 - Koa: 洋葱模型, 级联传递
- 响应机制
 - Express: res.send 立即响应
 - Koa: 设置ctx.body, 可累加, 经过中间件后响应

计算机网络

对比持续通信的方法？

轮询

- 通过 setInterval 或 setTimeout 定时获取并刷新页面上的数据
- 定时查询, 不一定有新数据
- 并发较多时, 增加服务器负担

长连接

- 页面其纳入 iframe, 将 src 设为长连接
- 减少无用请求
- 并发较多时, 增加服务器负担

长轮询

- 服务端收到请求后, hold 住链接, 直到新消息返回时才响应

- 减少无用请求
- 返回数据顺序无保证

Flash Socket

- 客户端通过嵌入 Socket 类 Flash与服务器端的 Socket 接口通信
- 真正即时通信
- 非 HTTP 协议，无法自动穿越防火墙

WebSocket

- 在客户端和服务端间打开交互式通信会话
- 兼容 HTTP 协议。与 HTTP 同属应用层。默认端口是 80 和 443
- 建立在 TCP 协议基础之上，与 HTTP 协议同属于应用层
- 数据格式轻量，性能开销小，通信高效
- 可以发送文本，也可以发送二进制数据
- 没有同源限制
- 协议表示符：ws，加密 wss

socket.io

- 跨平台的 WebSocket 库，API 前后端一致，可以触发和响应自定义事件

```
// 服务端
const io = require("socket.io")(3000)
io.on('connection', socket => {
  socket.on('update item', (arg1, arg2, callback) => {
    console.log(arg1, arg2)
    callback({ status: 'fulfilled' })
  })
})
// 客户端
const socket = io()
socket.emit('update item', "1", { name: 'updated' }, res => {
  console.log(res.status) // ok
})
```

网络结构按五层和七层分别是？

TCP / IP 体系结构

- 网络接口层
- 网际层 IP
- 运输层 TCP 或 UDP
- 应用层（TELNET FTP SMTP等）

五层

- 物理层
- 数据链路层
- 网络层
- 运输层
- 应用层

七层：Open System Interconnect Reference Model 开放式系统互联通信参考模型

- 物理层
- 数据链路层

- 网络层
- 传输层
- 会话层
- 表达层
- 应用层

什么是 TCP 三次握手，为什么要三次握手？

(1) 什么是 TCP 三次握手？

- 起初
 - 客户端 CLOSED
 - 服务端 CLOSED
- 第一次握手
 - 客户端发送请求报文
 - 传输自身通讯初始序号
 - 客户端 SYN-SENT
- 第二次握手
 - 服务器接收请求报文
 - 同意连接
 - 传输自身通讯初始序号
 - 服务端 SYN-RECEIVED
 - 拒绝连接
 - 服务端 REFUSED
- 第三次握手
 - 客户端接收应答报文
 - 客户端发送确认报文
 - 客户端 ESTABLISHED
 - 服务端接收确认报文
 - 服务端 ESTABLISHED

(2) 为什么要三次握手？

- 客户端首次发送请求无应答，TCP 启动超时重传
- 服务器收到重传的请求，建立连接，接收数据，关闭连接
- 服务器收到客户端首次发送请求，再次建立连接，但客户端已经关闭连接

需要三次握手，客户端发送确认报文后再建立连接，避免重复建立连接

浏览器有哪些请求方法？

安全：请求会影响服务器资源

幂等：多次执行重复操作，结果相同

方法	描述	请求体	响应体	支持表单	安全	幂等	可缓存
GET	请求资源	无	有	是	是	是	是
HEAD	请求资源头部	无	无	否	是	是	是
POST	发送数据 数据类型由 Content-Type 指定	有	有	是	否	否	响应头包含 expires 和 max-age
PUT	发送负载 创建或替换目标资源	有	无	否	否	是	否
DELETE	删除资源	不限	不限	否	否	是	否
CONNECT	创建点到点沟通隧道	无	有	否	否	否	否
OPTIONS	检测服务器支持方法	无	有	否	是	是	否
TRACE	消息环路测试 多用于路由检测	无	无	否	是	是	否
PATCH	部分修改资源	有	有	否	否	否	否

提交表单的内容类型有哪些？

- application/x-www-form-urlencoded：初始的默认值

```
Content-Type: application/x-www-form-urlencoded
...
key1=value1&key2=value2
```

- multipart/form-data：适用于使用 `<input>` 标签上传的文件

```
Content-Type: multipart/form-data; boundary=-----数据边界值
...
-----数据边界值
Content-Disposition: form-data; name="key1"
value1
-----数据边界值
Content-Disposition: form-data; name="key2"
value2
```

- text/plain：HTML5 引入类型

```
Content-Type: text/plain
...
key1=value1
key2=value2
```

docker 与虚拟机区别

启动速度

- Docker: 秒级启动
- 虚拟机: 几分钟启动

需要资源

- Docker: 操作系统级别虚拟化, 与内核直接交互, 几乎没有性能损耗
- 虚拟机:
 - 通过虚拟机监视器 (Virtual machine monitor, VMM, Hypervisor)
 - 厂商: Citrix XenServer, Hyper-V, 开源 KVM、Xen、VirtualBSD

轻量

- Docker: 内核与应用程序库可共享, 占用内存极小, 资源利用率高
- 虚拟机: 同样硬件环境, Docker 运行的镜像数量远多于 虚拟机数量

安全性

- Docker: 安全性更弱。租户 root 与宿主机 root 等同, 有提权安全风险
- 虚拟机:
 - 租户 root 权限与宿主机的 root 虚拟机权限分离
 - 用如 Intel 的 VT-d 和 VT-x 的 ring-1 硬件隔离技术, 虚拟机难以突破 VMM 直接与宿主或彼此交互

可管理型

- Docker: 集中化管理工具还不算成熟
- 虚拟机: 拥有相对完备的集中化管理工具

高可用和可恢复性

- Docker: 依靠 快速重新部署 实现
- 虚拟机: 负载均衡、高可用、容错、迁移、数据保护, VMware可承诺 SLA (服务级别协议) 99.999% 高可用

快速创建、删除

- Docker: 秒级别, 开发、测试、部署更快
- 虚拟机: 分钟级别

交付、部署

- Docker: 在 Dockerfile 中记录了容器构建过程, 可在集群中实现快速分发和快速部署
- 虚拟机: 支持镜像

对比 Intel 的 VT-x, VT-d, VT-c

VT-x

- 运行 ESXI 上的 64位 Guest OS 基本指令
- Intel 运用Virtualization 虚拟化技术中的指令集
- 减少 VMM 干预, 硬件支持 VMM 与 Guest OS
- 需要 VMM 干预, 更快速、可靠、安全切换
- 更灵活、更稳定

VT-d

- 使一个 Guest OS 独占并直接存取硬件设备，加快读写速度
- 开启条件：北桥芯片支持，BIOS 开启

VT-c

Virtual Machine Direct Connect

- 虚拟机的虚拟网络卡传送透过 VMM 来进行传输
- Guest OS 可以直接对 实体网络 I/O 进行存取
- 加强 VT-d，让多个虚拟机与实体 I/O 装置同时建立通道

Virtual Machine Direct Queues

- 由 VMM 管理的虚拟化 Switch 转送封包给虚拟机
 - 封包流向哪个虚拟机，需要额外 CPU 资源
- 透过网卡内建 Layer2 classifier / sorter 加速网络资料传送
 - 在芯片里安排并通过队列排序好封包给虚拟机，不需要 VMM 支持
 - 大大降低网络负载和 CPU 使用率

对比 Cookie、LocalStorage、SessionStorage、Session、Token

Cookie

- 保存位置：客户端
- 生命周期：expires 前有效，不设置 会话期间有效
- 存储大小：一般 4KB
- 数据类型：字符串
- 特点：
- 请求头 cookie，同域名每次请求都附加
- 响应头 set-cookie 设置 cookie

LocalStorage

- 保存位置：客户端
- 生命周期：不清空缓存，一直有效
- 存储大小：一般 5MB
- 数据类型：字符串

SessionStorage

- 保存位置：客户端
- 生命周期：会话期间有效
- 存储大小：一般 5MB
- 数据类型：字符串

Session

- 保存位置：服务端，sessionid 存在 cookie 或 跟在 URL 后
- 生命周期：默认 20分钟，可更改。清空 cookie 或 更改 URL 传参，影响 Session
- 存储大小：不限制
- 数据类型：字符串、对象、数组，序列化后以字符串存储

Token

- 保存位置：客户端 Cookie / WebStorage / 表单 等
- 生命周期：根据存储位置决定
- 存储大小：根据存储位置决定
- 数据类型：字符串

- 组成
- uid 用户身份标识
- time 时间戳
- sign 签名
- 其他参数

JWT - JSON Web Token

特点

- 将部分用户信息存储本地，避免重复查询数据库

组成

Header (头部)

- alg 表示签名的算法，默认是 HMAC SHA256 (HS256)
- typ 表示令牌的类型，JWT

Payload (负载)

- 7 个官方字段
 - jti 编号
 - iss 签发人
 - sub 主题
 - aud 受众
 - exp 过期时间
 - nbf 生效时间
 - iat 签发时间
- 私有字段
 - name 姓名
 - admin 是否管理员

Signature (签名)

- secret 密钥
- 使用指定 alg 签名算法，生成签名
 - HMAC SHA256(btoa(Header) + '.' + btoa(Payload), secret)

算出签名

什么是 Service Worker?

- Web 应用程序、浏览器与网络之间的代理服务器
- 拦截网络请求，并根据网络采取适当动作，更新服务器资源
- 完全异步，不阻塞 JavaScript 线程，不能访问 DOM
- 提供入口以推送通知和访问后台同步 API
- 用于离线缓存，后台数据同步，响应来自其它资源的请求，集中接收成本高的数据更新

性能

如何测量并优化前端性能？

如何测量并优化前端性能？

(1) 前端页面加载阶段

网络

- 重定向
 - redirectStart
 - redirectEnd
- DNS 查询
 - domainLookupStart
 - domainLookupEnd
- TCP 连接
 - connectStart
 - connectEnd

请求响应

- 发送请求
 - requestStart
- 收到响应
 - responseStart
 - responseEnd

解析渲染

- DOM 解析
 - domLoading
 - domInteractive
- DOM 渲染
 - domContentLoadedEventStart
 - domContentLoadedEventEnd
 - domComplete

(2) 衡量性能的时间点

- 首字节时间：收到服务器返回第一个字节的时间。反映网络后端的整体响应耗时
- 白屏时间：第一个元素显示时间
- 首屏时间：第一屏所有资源完整显示时间。SEO 指标，百度建议2秒内为宜，不超过3秒

** (3) 测量性能的工具

Chrome 浏览器，开发者工具

- Performance：通过录制页面加载过程，可以获得加载，脚本执行，渲染，布局等时间分布情况
- Lighthouse：可以分别模拟移动端和电脑端打开URL，从性能、可访问性、最佳实践（安全、用户体验、函数是否符合标准等）、SEO、单页应用等角度，获得修改建议

NodeJS

- SiteSpeed：

```
npm i -g sitespeed.io && sitespeed.io https://leetcode-cn.com
```

线上测试工具

- Speedcurve: <https://speedcurve.com>
- webpagetest: <https://www.webpagetest.org>

(4) 性能优化方法

DNS 预读取:

```
<link rel="dns-prefetch" href="//leetcode-cn.com" />
```

预加载

- prefetch: `<link rel="prefetch" href="//leetcode-cn.com" />`
- Image / Audio

懒加载

- 图片懒加载
- DOM懒加载

渲染

- 减少重排
- 减少重绘
- 节流
- 防抖

缓存

- 离线缓存
- HTTP 缓存

图片

- 合并图片请求
 - Base64 将图片嵌入 CSS
 - SVG sprites
- 响应式图片: 不同分辨率和 DPR 下显示不同图片
 - 媒体查询
 - IMG 的 srcset 属性
- 图片压缩
 - image-webpack-loader、imagemin-webpack-plugin 工程化压缩图片工具
 - WEBP 和 AVIF 图片格式

HTTP

- 使用 HTTP2 / HTTP3
- 使用 CDN
- 使用 gzip / br 压缩静态资源

SEO

前端中有哪些 SEO 优化手段?

文字

- 字号 14px 以上, 16px 为宜, 避免小于 12px, 有行距、换行和段落
- 避免使用 visibility:hidden, display: none, 绝对定位, 与背景相同颜色, 堆砌关键词, 隐藏主要内容

图片

- Logo 的 DIV 中写关键词，设置 text-indent 为负数隐藏
- 图片添加 width 和 height，避免读取元数据，再重排
- 图片添加 alt，描述图片内容，便于搜索引擎蜘蛛理解，提升可访问性
- 主要图片与内容相关，不要过长或过窄，正方形为宜
- 主要图片设置 src 属性，避免全部图片都使用懒加载
- 图片可以点击放大，多张图片间可以切换

适配

独立 H5 网站

- H5移动版与PC版域名不同，移动版子域名，推荐使用 m.x.com 等移动网站常用域名
- 向搜索引擎提交适配规则：PC版和H5移动版组成——对应的URL对或正则匹配关系
- 不同域名，使用不同蜘蛛抓取

代码适配

- H5 移动版和PC版域名相同，返回代码不同
- 后端根据 User-Agent，返回不同的代码
- 添加响应头：Vary: User-agent
- 同一URL，蜘蛛应使用不同UA或不同蜘蛛多次抓取

自适应

- H5 移动版和PC版 域名相同，返回代码相同
- 前端通过相对单位、媒体查询、srcset 实现响应式或自适应布局
- 同一URL，蜘蛛可以只抓取一次

SSR: Server Side Render 服务端渲染

安全

什么是帆布指纹?

目标：追踪用户

背景：移动互联网 IP 经常变化，用户禁用 cookie

解决：

Canvas Fingerprinting

- 不通过 cookie
- 用户不易屏蔽

手写代码

```
const canvas = document.createElement('canvas')
const ctx = canvas.getContext('2d')
const txt = 'canvasfingerprinting'
ctx.fillText(txt, 0, 0)
const b64 = canvas.toDataURL().replace('data:image/png;base64,', '') // 返回一个包含图片的 data URI，并替换 Mine-type，返回 base64 编码
const bin = atob(b64) // 函数能够解码 base64 编码的字符串数据
const canvasFingerprinting = bin2hex(bin.slice(-16, -12)) // 将 unicode 编码转为十六进制
```

fingerprintJS2 / fingerprintJS

除 canvas 外，加入其它影响因素：

- navigator.userAgent 用户代理
- navigator.language 用户语言
- screen.colorDepth 屏幕色彩信息
- screen.height / screen.width 屏幕宽度和高度
- Date().getTimezoneOffset() 格林威治时间与本地时间时差
- sessionStorage 是否支持 sessionStorage
- localStorage 是否支持 localStorage
- indexedDB 是否支持 indexedDB
- addBehavior 是否支持 document.body.addBehavior IE5 属性
- openDatabase 是否支持调用本地数据库
- cpuClass 浏览器所在系统的 CPU 等级
- navigator.platform 客户端操作系统
- doNotTrack 是否支持 Do not track 功能
- Flash plugin / Adobe PDF reader / QuickTime / Real players / ShockWave player / Windows media player / Silverlight / Skype

对比各种分布式 ID 的生成方式？

UUID

- 生成简单，本地生成
- 无序，无意义

数据库自增ID

- auto_increment 实现简单，单调自增，查询快
- 数据库服务器宕机风险
 - 使用 SELECT LAST_INSERT_ID() 代替 max(id) 提高高并发时的查询效率
 - 不同数据库服务器，设置不同自增起始值和步长

数据库自增ID段

```
CREATE TABLE ids_increment (  
  type int(8) NOT NULL COMMENT '业务类型',  
  max_id int(8) NOT NULL COMMENT '当前业务的最大ID',  
  step int(8) NOT NULL COMMENT '当前业务的ID段步长',  
  version int(8) NOT NULL COMMENT '当前业务的版本号，每次分配 +1，乐观锁'  
)
```

Redis

- set 字段的初始值 incr 该字段，每次 +1
- 使用 RDB 定时持久化
- 使用 AOF 命令持久化

雪花算法

- 64 位 ID：符号位（1 位）+ 时间戳（41 位）+ 机器 ID（5 位）+ 数据中心 ID（5 位）+ 自增值（12 位）

列举各种跨域方法

(1) 跨域

协议、域名、端口不同，需要跨域

(2) 跨域解决方案

jsonp: JSON with Padding

- <script> 标签没有同域限制
- 服务端返回页面上 callback 包裹的 json 数据
- 兼容性好
- 仅支持 GET

CORS: Cross-origin resource sharing 跨域资源共享

请求类型:

简单请求:

满足以下 2 条件

- GET HEAD POST
- Content-Type
 - application/x-www-form-urlencoded
 - multipart/form-data
 - text/plain

**** 复杂请求** :**

不满足以上2条件

- 需发送 OPTION 预检请求，查询服务器支持的方法

按请求类别处理

```
const express = require('express')
const app = express()
const whiteList = ['http://localhost']
app.use((req, res, next) => {
  const origin = req.headers.origin
  if (whiteList.includes(origin)) {
    // 允许访问源，附带凭证请求时，必须指定 URI。相反，可设置为 *
    res.setHeader('Access-Control-Allow-Origin', origin)
    // 允许请求头，逗号隔开
    res.setHeader('Access-Control-Allow-Headers', 'authorization, username')
    // 允许请求方法，逗号隔开
    res.setHeader('Access-Control-Allow-Methods', 'GET, HEAD, POST, OPTIONS')
    /* 允许凭证
       客户端需配置 const xhr = new XMLHttpRequest()
       xhr.withCredentials = true */
    res.setHeader('Access-Control-Allow-Credentials', true)
    // 预检请求的返回结果，允许请求头和请求方法可以被缓存多久，缓存期间不再发送预检请求，单位秒
    res.setHeader('Access-Control-Allow-Max-Age', 10)
    // 允许响应头
    res.setHeader('Access-Control-Expose-Headers', 'authorization, username')
    if (req.method === 'OPTIONS') {
      res.end() // 预检请求直接返回空响应体
    }
  }
})
```

```
}  
})
```

postMessage

- window属性
- 跨窗口，跨框架 frame、iframe，跨域
- 允许不同源脚本采用异步方式进行有限通信

```
otherWindow.postMessage(message, targetOrigin, [transfer])  
// otherWindow:  
iframe.contentWindow 属性  
window.open 返回  
window.frames 访问  
window.onmessage = e => console.log(e.data) // 接收返回数据
```

Websocket

- 基于 TCP
- 全双工通信
- 应用层协议

```
// 客户端  
const socket = new WebSocket('ws://localhost:3000')  
socket.onopen = () => socket.send('message') // 向服务器发送数据  
socket.onmessage = e => console.log(e.data) // 接收服务器返回数据  
// server.js  
const express = require('express')  
const app = express()  
const ws = require('ws')  
const wss = new ws.Server({port: 3000})  
wss.on('connection', ws => {  
  ws.on('message', data => console.log(data))  
})
```

代理服务器转发

- 代理服务器设置 CORS 请求头字段
 - nginx
 - add_header
 - kangle
 - 回应控制，增加表，标记模块，add_header
 - nodejs
 - response.writeHead
- 代理服务器向源服务器继续请求
 - nginx
 - 七层 HTTP 代理：配置 http

```
http {
    server {
        listen 80;
        location / {
            proxy_pass http://127.0.0.1:3000
        }
    }
}
```

四层 TCP 代理: 配置 stream

```
stream {
    upstream proxyServer {
        server 127.0.0.1:3000;
    }
    server {
        listen 80;
        proxy_pass proxyServer;
    }
}
```

- kangle
 - 七层 HTTP 代理
 - 请求控制, 增加表
 - 目标设置为代理
 - 匹配模块设置原域名和端口, 标记模块设置目标域名和端口
- nodejs
 - 七层 HTTP 代理

```
const http = require('http')
http.request({
  host: '127.0.0.1',
  port: 3000,
  url: '/'
})
```

document.domain

- 主域名相同, 子域名不同
- 设置值为 主域名

iframe

- window.name
 - 不超过 2MB
 - 同一个iframe
 - 加载跨域页, 跨域页设置 window.name
 - 加载同域页, 读取 iframe.contentWindow.name
- location.hash
 - 不会被 URL 编码
 - 加载跨域页, 参数跟在 # 后。window.onhashchange 监听 hash 变化
 - 跨域页获取 location.hash, 加载同域页, 参数跟在 # 后

- 同域页设置 `window.parent.parent.location.hash = location.hash`，将参数回传

数据库

什么是乐观锁，什么是悲观锁

并发控制

- 需要保证并发情况下的数据准确性
- 保证一个用户的工作不会对另一个用户的工作产生不合理的影响
- 避免
 - 脏读：一个事务看见另一个事务未提交的数据
 - 不可重复读：一个事务两次读取到的数据内容不同，另一事务修改了数据
 - 幻读：一个事务两次读取到的数据条数不同，另一事物增删了数据

乐观锁：多读 适用

- 仅在数据提交更新时，检测数据冲突。冲突时，采用竞争机制
- 依靠数据本身来保证数据正确性，如添加字段 `version`

悲观锁：多写 适用

- 共享锁（读锁）：多个事务同一数据共享锁，只读不可写
- 排他锁（写锁）：一次仅一个事务可读写数据
 - 依赖数据库的锁机制，如 MySQL：`select ... for update`

什么是事务

(1) 定义

访问并可能更新数据库中数据项的一个程序执行单元 (Unit)

一条 SQL、一组 SQL 和 整个程序都可以是事务

(2) 特点

- 原子性 (atomicity)：不可分割，要么都做，要么都不做
- 一致性 (consistency)：一个一致性状态变到另一个一致性状态
- 隔离性 (isolation)：一个事务的执行不能被其他事务干扰，操作和数据使用不依赖和干扰其它并发事务
- 持久性 (durability)：提交后的改变永久生效。不受故障或其它操作影响

(3) 在 Mysql 中使用事务

```
# 关闭自动提交
SET autocommit = 0
# 或者开始一个事务
BEGIN
若干SQL语句
# 提交事务
COMMIT 对数据库所有修改都永久生效
若干SQL语句
# 回滚事务
ROLLBACK 结束当前事务，撤销所有未提交的更改
# 设置保存点
SAVEPOINT id 创建名为 id 的保存点
# 回滚事务到保存点
```

```
ROLLBACK TO id 把事务回滚到保存标记点
# 设置事务的隔离级别
SET TRANSACTION InnoDB 存储引擎提供事务的隔离级别有
(1) READ UNCOMMITTED
(2) READ COMMITED
(3) REPEATABLE READ
(4) SERIALIZABLE
```

项目管理

您使用过哪些文档管理工具？

API 接口协作管理工具

- Swagger
- Spring
- Apizza

文档协作工具

- 石墨文档
- 有道笔记 + 有道云协作
- 印象笔记

自动文档生成

- apiDoc: 根据 API 描述, 自动生成文档
- jsDoc: 根据 JS 注释, 自动生成文档

对比 Git 和 SVN?

架构

- Git:
 - 分布式, 本地克隆版本库
 - 无网络依然可以操作分支, 提交文件, 查看历史记录
- SVN:
 - 集中式, 代码一致性高
 - 依赖网络

存储

- Git: 按元数据, 即描述文件的数据
- SVN: 按文件

分支

- Git: 移动指针, 创建切换快
- SVN: 拷贝目录

权限

- Git:
 - 经常按项目配置
 - 更适合开源
- SVN:
 - 经常按目录配置

- 更适合内部开发